LABORATORIO 3

Sistema de inventario para una aduana

Investigación: bases de datos relacionales, Supabase y agentes de IA con n8n

Curso: Bases de Datos
Autor: Reymond Salguero
Fecha: 23 de octubre de 2026

Plataforma elegida: Supabase
Motor y dialecto SQL: PostgreSQL

Alcance: seis páginas de contenido, más portada y referencias. Los ejemplos son académicos; el esquema definitivo, los datos de prueba y el workflow corresponden a las fases posteriores.

A. Bases de datos y SQL

Organizar información con reglas

Entiendo una base de datos como un conjunto organizado de información que puedo consultar y relacionar. En una aduana, esto me permite seguir una mercancía desde su declaración hasta su ubicación en bodega. El sistema gestor de bases de datos (SGBD) es el software que administra ese conjunto: ejecuta consultas, controla permisos y protege la consistencia. Para mí, la base de datos es el archivo organizado y PostgreSQL es quien aplica sus reglas de acceso y almacenamiento (PostgreSQL Global Development Group, s. f.-c).

En un modelo relacional distribuyo los datos en tablas vinculadas por claves. En uno no relacional puedo encontrar documentos, pares clave-valor o grafos, según el producto. No interpreto NoSQL como ausencia de reglas ni como una opción siempre más rápida: la elección depende de las consultas y de la estructura del problema. Para este laboratorio prefiero relaciones explícitas porque necesito conectar importadores, declaraciones, productos e inspecciones sin perder su correspondencia.

Tablas, filas y tipos de datos

Veo una tabla como un conjunto de registros de una misma clase. En producto, cada fila representa un producto y cada columna describe un atributo, como nombre o peso. Una celda contiene el valor de ese atributo para una fila. Evito guardar cantidades y fechas como texto: aunque visualmente parezcan correctas, después dificultan los cálculos y las validaciones.

Usaría INTEGER o BIGINT para identificadores y cantidades enteras; NUMERIC(14,2) para importes que requieran exactitud decimal; VARCHAR o TEXT para nombres; DATE para una fecha sin hora; TIMESTAMPTZ para eventos con referencia temporal, y BOOLEAN para condiciones de sí o no. Guardaría códigos arancelarios como texto para conservar ceros iniciales. También documentaría moneda y unidad de medida: un número aislado no me dice si representa dólares, kilogramos o unidades (PostgreSQL Global Development Group, s. f.-b).

Identificar sin depender del nombre

La llave primaria (PK) identifica cada fila y no admite duplicados ni valores nulos. La llave foránea (FK) vincula una fila con una clave válida de otra tabla, o de la misma tabla. Una llave compuesta utiliza varias columnas; por ejemplo, (id_declaracion, numero_linea) identifica una línea dentro de su declaración. No elegiría el nombre del importador como PK, porque puede cambiar o repetirse (PostgreSQL Global Development Group, s. f.-a).

Para mí, separar identificación y descripción evita confundir dos empresas con nombres parecidos. También facilita actualizar un nombre sin modificar todas las relaciones que apuntan a esa empresa.

A. Relaciones, integridad y operaciones

Cardinalidades aplicadas a la aduana

En una relación 1:N, un importador puede tener varias declaraciones y cada declaración referencia un importador. Coloco la FK en declaracion_aduanera. En una N:M, una declaración contiene varios productos y un producto aparece en varias declaraciones; por eso utilizo detalle_declaracion, con referencias a ambos extremos y atributos propios como cantidad y valor unitario. Así no necesito una columna diferente para cada producto (Watt, 2014a).

Para ilustrar 1:1 propongo una ficha operativa por contenedor: ficha_contenedor.id_contenedor sería PK y FK a la vez. Esto limita a una ficha por contenedor, pero permite que un contenedor todavía no tenga ficha. Si exijo participación obligatoria en ambos extremos necesito una validación adicional; una FK UNIQUE por sí sola no la garantiza. La relación recursiva aparece cuando funcionario.id_supervisor referencia funcionario.id_funcionario. Prohibir que alguien se supervise a sí mismo no elimina ciclos de varias personas; esos ciclos requieren otra validación.

Integridad y restricciones

La integridad referencial impide asociar una declaración a un importador inexistente. NOT NULL obliga a informar un atributo; UNIQUE evita repeticiones; CHECK valida condiciones como cantidad > 0; DEFAULT aporta un valor cuando no lo especifico. Un CHECK puede aceptar una expresión desconocida por NULL, de modo que lo combino con NOT NULL cuando el dato es obligatorio. Tampoco usaría CHECK para comparar sumas entre tablas: reservaría esa regla para otra estrategia de validación (PostgreSQL Global Development Group, s. f.-a).

DDL, DML y lectura de información

Con DDL defino la estructura; con DML inserto, actualizo y elimino datos. SELECT consulta información y algunas clasificaciones lo separan como DQL. Comprendo la diferencia pensando en una bodega: CREATE construye los estantes, INSERT incorpora fichas y SELECT me permite revisarlas. Los siguientes ejemplos forman una demostración independiente, no el script final del laboratorio (PostgreSQL Global Development Group, s. f.-c).

```
CREATE TEMP TABLE ejemplo_producto (
  id INTEGER PRIMARY KEY, nombre TEXT NOT NULL,
  cantidad INTEGER NOT NULL CHECK (cantidad >= 0)
);
INSERT INTO ejemplo_producto VALUES (1, 'Monitor', 10);
SELECT nombre, cantidad FROM ejemplo_producto;
UPDATE ejemplo_producto SET cantidad = 9 WHERE id = 1;
DELETE FROM ejemplo_producto WHERE id = 1;
```

Un JOIN combina filas relacionadas. Por ejemplo: SELECT d.numero_dua, i.nombre FROM aduana.declaracion_aduanera d JOIN aduana.importador i ON i.id_importador = d.id_importador; dependerá de las columnas definitivas. Usaría LEFT JOIN si necesito conservar declaraciones sin coincidencia opcional. Antes de sumar valores revisaría la multiplicación de filas por cada detalle (PostgreSQL Global Development Group, s. f.-d).

A. Normalización y representación del modelo

Evitar duplicaciones que producen errores

Al normalizar busco que cada dato dependa de la clave que le corresponde. Si mezclara importador, teléfono, producto y cantidad en una hoja enorme, repetiría los datos del importador por cada mercancía. Cambiar un teléfono exigiría editar muchas filas y podría dejar versiones contradictorias. Ese riesgo me ayuda a entender por qué dividir tablas tiene una finalidad práctica (Watt, 2014c).

Primera forma normal (1FN). Mantengo un valor por atributo según el dominio y elimino grupos repetidos. En lugar de escribir “monitor, teclado, ratón” en una celda, represento cada línea de mercancía como un registro de detalle_declaracion. Cada fila conserva su identificación y una cantidad interpretable.

Segunda forma normal (2FN). Primero cumplo 1FN y elimino dependencias parciales respecto de cualquier clave candidata compuesta. Si la clave del detalle es (id_declaracion, numero_linea), la fecha de la declaración depende solo de id_declaracion: la ubico en la cabecera. La cantidad de una línea sí corresponde a la línea completa.

Tercera forma normal (3FN). Evito dependencias transitivas problemáticas de atributos no clave. Si un producto referencia id_categoria y este determina la descripción de la categoría, guardo esa descripción en categoria_arancelaria. No la copio a cada producto. Así reduzco anomalías de inserción, modificación y eliminación sin asumir que normalizar garantiza por sí solo un buen rendimiento (Watt, 2014c).

El DER como contrato antes del SQL

El Diagrama Entidad-Relación me sirve como plano: identifica entidades, atributos y asociaciones antes de crear tablas. En notación Chen encuentro entidades como rectángulos, relaciones como rombos y atributos como óvalos. Pata de Gallo utiliza extremos gráficos para indicar uno o muchos; también permite mostrar si una participación es opcional u obligatoria. Elegiría esta última para acercar el diagrama al esquema relacional (Watt, 2014b).

Para que el plano sea útil, comprobaría que el DBML y el SQL coincidan en nombres, tipos, PK, FK y nulabilidad. También revisaría la granularidad: inventario_bodega debe conservar a qué línea declarada pertenece una existencia, además de su bodega. Si guardara únicamente producto y bodega, perdería la capacidad de distinguir mercancía retenida de otra liberada del mismo producto.

Mantendría separados el país de origen de la mercancía, el país del importador y la procedencia del contenedor. En mi interpretación son datos distintos: comprar a una empresa de un país no demuestra que el producto se fabricó allí. Esta decisión del modelo académico evita respuestas ambiguas cuando pregunte por mercancías de China.

B. Gestores y elección de Supabase

Comparé tres gestores por su arquitectura, administración y ajuste al laboratorio. Mi elección no depende de que uno sea universalmente superior; depende de cómo quiero consultar información compartida y limitar el acceso del agente.

Criterio	SQLite	MySQL	PostgreSQL
Arquitectura	Motor embebido; archivo local	Servidor de BD	Servidor de BD
Administración	Muy reducida en uso local	Usuarios y permisos en servidor	Roles, esquemas y privilegios
Uso apropiado	Apps locales y prototipos ligeros	Aplicaciones web y datos compartidos	Datos relacionados y consultas complejas
Concurrencia	Un escritor a la vez; varios lectores	Transacciones con InnoDB	Transacciones y MVCC
Ajuste a mi caso	Útil para aprender sin servidor	Alternativa válida; nodo n8n disponible	Motor de Supabase; nodo Postgres en n8n

Basé la comparación en la documentación de SQLite, Oracle y PostgreSQL. SQLite simplifica el almacenamiento local, mientras que los servidores facilitan centralizar conexiones y permisos. No descartaría MySQL para un sistema aduanero; para este proyecto ya decidí trabajar con Supabase y, por tanto, con PostgreSQL (SQLite, s. f.; Oracle, s. f.; PostgreSQL Global Development Group, s. f.-c).

Supabase es la plataforma; PostgreSQL es el motor

Usaré Supabase para alojar y administrar la base de datos. Esto conserva las tablas, claves y consultas de PostgreSQL; no lo convierte en otro dialecto SQL. Elegiría un esquema llamado aduana para aislar mis objetos de los componentes de la plataforma. Ejecutaría las migraciones desde SQL Editor, sin crear otra base de datos ni modificar los esquemas internos auth o storage.

Conectaría n8n mediante su nodo Postgres. Copiaría host y parámetros desde Connect en Supabase. Para una red solo IPv4, usaría Session pooler en el puerto 5432; el usuario de un rol propio tendría el formato agente_n8n_readonly.<PROJECT_REF>. En conexión directa usaría el nombre del rol sin ese sufijo, sujeto a conectividad disponible. Configuraría TLS y validación del certificado según la conexión documentada, sin confundir la contraseña de PostgreSQL con una API key (Supabase, s. f.-a).

C. n8n y agentes de inteligencia artificial

Automatizar tareas y consultar con herramientas

Entiendo n8n como una plataforma que conecta pasos de trabajo mediante workflows. Un nodo representa una operación; un trigger inicia la ejecución ante un evento, y una credencial guarda los datos de autenticación que necesita una integración. Para mi caso, Chat Trigger recibiría la pregunta y AI Agent coordinaría la consulta. Las credenciales se configurarían en la instancia, fuera del archivo JSON entregable.

Un LLM es un modelo de lenguaje que genera respuestas a partir del contexto. Una tool es una capacidad externa descrita al modelo, como consultar existencias. Un agente puede elegir herramientas y utilizar sus resultados para responder; una conversación tradicional puede limitarse a generar texto. La diferencia es de capacidades y configuración, porque un chatbot también puede integrar herramientas. El nodo AI Agent necesita un modelo compatible y al menos una herramienta conectada (n8n, s. f.-a).

Conectaría el modelo al puerto ai_languageModel, la memoria a ai_memory y la herramienta de consulta a ai_tool. Simple Memory me permitiría mantener un contexto reciente por sessionId, por ejemplo para entender “¿y en la otra bodega?”. No la tomaría como fuente de stock: consultaría la base de datos en cada respuesta operativa. Tampoco usaría Simple Memory en queue mode, porque las llamadas pueden llegar a trabajadores distintos (n8n, s. f.-c).

Seguridad aplicada en la base de datos

Aplicaría mínimo privilegio con el rol agente_n8n_readonly: LOGIN sin capacidades administrativas, CONNECT a la base, USAGE del esquema aduana y SELECT únicamente en los objetos autorizados. No le daría propiedad de tablas, membresía en roles potentes ni BYPASSRLS. Los privilegios de futuras tablas dependerían del rol que las cree, por lo que también documentaría ALTER DEFAULT PRIVILEGES para el propietario correcto (PostgreSQL Global Development Group, s. f.-e).

El prompt del agente no reemplaza esos permisos. Preferiría herramientas con SELECT fijo y parámetros para las preguntas del laboratorio. El nodo Postgres admite parámetros de consulta; los valores se enviarían separados del texto SQL, sin concatenar lo escrito por el usuario. Si permito SQL generado, añadiría validación externa, lista de objetos permitidos, límite de filas y timeout. Un SELECT arbitrario todavía puede invocar funciones o consumir recursos (n8n, s. f.-b).

En Supabase distinguiría los permisos SQL de las políticas RLS, que limitan filas para los roles sujetos a ellas. Si activo RLS, escribiría políticas SELECT explícitas para el rol del agente y probaría el acceso con esa identidad. No usaría service_role como credencial de consulta: puede omitir RLS. Mantendría aduana fuera de los esquemas expuestos por la Data API si esa API no es necesaria (Supabase, s. f.-b, s. f.-c).

D. Dominio aduanero y E. Uso de IA

Qué información necesito representar

Comprendo la aduana como la organización encargada de controlar el ingreso y salida de mercancías, aplicar las disposiciones correspondientes y facilitar el comercio bajo control. En Costa Rica, el agente aduanero es un auxiliar autorizado de la función pública que representa a su mandante en los trámites que corresponden; no lo confundo con el funcionario que realiza una inspección (Asamblea Legislativa de Costa Rica, 1995).

La declaración aduanera comunica los datos necesarios para someter mercancías a un régimen. En el laboratorio usaré la denominación DUA, frecuente en la documentación del sistema TICA. La separaré en cabecera y líneas: la cabecera identifica la operación y cada detalle describe mercancías, cantidades y valores. El identificador DUA-2026-0001 será ficticio y didáctico, sin presentarlo como formato oficial de numeración (Ministerio de Hacienda, 2025).

El Sistema Armonizado organiza internacionalmente las mercancías; distingo la partida de cuatro dígitos y la subpartida de seis, con posibles extensiones nacionales. El arancel es un derecho aplicable según clasificación, origen y régimen. No asumiría que una tasa inventada sirve para liquidar impuestos reales: mis códigos y tasas de prueba deben identificarse como datos académicos (Organización Mundial de Aduanas, s. f.).

En mi modelo, importación representa el ingreso bajo el régimen correspondiente y exportación la salida; tránsito permite traslado bajo control aduanero y depósito mantiene mercancías almacenadas bajo ese control conforme a sus condiciones. “Retenida”, “Liberada” y “En Inspeccion” describirán estados operativos del ejercicio, no regímenes. Esta separación me permite registrar mercancía en depósito y, a la vez, señalar que está en inspección. El alcance es conceptual y no sustituye procedimientos oficiales (Asamblea Legislativa de Costa Rica, 1995).

Bitácora reflexiva y límites de lo realizado

En esta preparación utilicé ChatGPT para organizar la investigación y adaptar el proyecto a Supabase. La IA me ayudó a relacionar conceptos con el inventario aduanero y a localizar documentación. Entiendo que este texto es un borrador asistido que debo revisar y explicar con mis palabras. Todavía no he ejecutado el script definitivo ni probado el workflow; por eso no presento depuración o resultados de pruebas como experiencias realizadas.

Ejemplo propuesto 1: “Explícame por qué una declaración no puede apuntar a un importador inexistente”. El aprendizaje que busco es comprender la integridad referencial y el orden de inserción de tablas padre e hijas, antes de intentar desactivar una FK.

Ejemplo propuesto 2: “Propón datos ficticios coherentes para mercancía china retenida en dos bodegas”. Me permitiría comprobar cantidades, origen y trazabilidad. Aceptaría los datos solo después de verificar claves, unidades y reglas; no los trataría como operaciones reales.

Ejemplo propuesto 3: “Guíame para conectar n8n a Supabase con un rol SELECT y comprobar que UPDATE falla”. Mi objetivo es aprender a separar contraseña SQL, conexión y permisos. Registraré los errores reales, correcciones y evidencias cuando ejecute las fases posteriores, sin inventar capturas ni resultados.

F. Referencias

Asamblea Legislativa de Costa Rica. (1995). Ley General de Aduanas (Ley N.º 7557). Sistema Costarricense de Información Jurídica.
https://sinalevi.go.cr/ResultadosNormativa/Informacion?param1=25886&param2=131179&param3=1

Ministerio de Hacienda. (2025). Formato de mensaje del DUA (versión 3.17).
https://www.hacienda.go.cr/docs/Mensaje_TD_DUA-V3-17-12-03-2025.pdf

n8n. (s. f.-a). AI Agent. n8n Docs.
https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/n8n-nodes-langchain.agent/

n8n. (s. f.-b). Postgres node documentation. n8n Docs.
https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.postgres/

n8n. (s. f.-c). Simple Memory. n8n Docs.
https://docs.n8n.io/integrations/builtin/cluster-nodes/sub-nodes/n8n-nodes-langchain.memorybufferwindow/

Oracle. (s. f.). MySQL 8.4 reference manual: General information.
https://dev.mysql.com/doc/refman/8.4/en/introduction.html

Organización Mundial de Aduanas. (s. f.). What is the Harmonized System (HS)?
https://www.wcoomd.org/en/topics/nomenclature/overview/what-is-the-harmonized-system.aspx

PostgreSQL Global Development Group. (s. f.-a). Constraints. PostgreSQL documentation.
https://www.postgresql.org/docs/18/ddl-constraints.html

PostgreSQL Global Development Group. (s. f.-b). Data types. PostgreSQL documentation.
https://www.postgresql.org/docs/18/datatype.html

PostgreSQL Global Development Group. (s. f.-c). The SQL language. PostgreSQL documentation.
https://www.postgresql.org/docs/18/tutorial-sql.html

Referencias (continuación)

PostgreSQL Global Development Group. (s. f.-d). Joins between tables. PostgreSQL documentation. https://www.postgresql.org/docs/18/tutorial-join.html

PostgreSQL Global Development Group. (s. f.-e). Privileges. PostgreSQL documentation. https://www.postgresql.org/docs/18/ddl-priv.html

SQLite. (s. f.). Appropriate uses for SQLite. https://www.sqlite.org/whentouse.html

Supabase. (s. f.-a). Connect to your database. Supabase Docs. https://supabase.com/docs/guides/database/connecting-to-postgres

Supabase. (s. f.-b). Row level security. Supabase Docs. https://supabase.com/docs/guides/database/postgres/row-level-security

Supabase. (s. f.-c). Securing your data. Supabase Docs. https://supabase.com/docs/guides/database/secure-data

Watt, A. (2014a). The entity relationship data model. En Database design (2.ª ed.). BCcampus. https://opentextbc.ca/dbdesign01/chapter/chapter-8-entity-relationship-model/

Watt, A. (2014b). ER modelling. En Database design (2.ª ed.). BCcampus. https://opentextbc.ca/dbdesign01/chapter/chapter-10-er-modelling/

Watt, A. (2014c). Normalization. En Database design (2.ª ed.). BCcampus. https://opentextbc.ca/dbdesign01/chapter/chapter-12-normalization/

Documentación consultada el 23 de octubre de 2026. La versión de PostgreSQL del proyecto Supabase debe comprobarse al implementar mediante SELECT version().